

FITNESSPOT

Adrian SPYCHER



Francisco GARCIA

Travail Pratique Individuel – CFPT

20 MAI 2021

Table des matières

1 Introduction 3
2 Cahier des charges 3
2.1 Organisation 3
2.2 Titre 3
2.3 Résumé du travail 3
2.4 Matériel et logiciels à disposition 3
2.5 Prérequis 4
2.6 Descriptif complet du projet 4
2.6.1 Planification 4
2.6.2 Méthodologie 4
2.6.3 Description de l'application 4
2.6.4 Prototype d'interface utilisateur 4
2.6.5 MCD 6
2.7 Livrables 7
2.8 Points techniques évalués spécifiques au projet (obligatoire) correspondants aux points A14 à A20 du formulaire d'évaluation 7
3 Méthodologie 8
3.1 S'informer 8
3.2 Planifier 8
3.3 Décider 9
3.4 Réaliser 9
3.5 Contrôler 9
3.6 Évaluer 10
4 Planification 10
4.1 Product Backlog 10
4.2 Planning 14
5 Généralités concernant l'implémentation 16
5.1 Librairies et outils externes utilisés 16
5.2 Architecture du projet 22
5.2.1 Modèle Vue Contrôleur 22
5.2.2 Arborescence 23
5.2.3 Diagramme de navigation 24
5.3 Base de données 24

5.4 Fonctionnalités intégrées 25
6 Analyse des fonctionnalités majeures 27
6.1 Singleton de connexion à la base de données 27
6.2 Mes modèles, les Prepared Statements et les Bound Parameters 28
6.3 Récupération des spots pour la map 30
6.4 Barre de recherche 31
6.5 Pagination 33
7 Tests 37
7.1 Méthodologie de test 37
7.2 Scénarios de test 38
7.3 Évolution des tests 48
8 Conclusion 49
8.1 Difficultés rencontrées 49
8.2 Amélioration possible 49
8.3 Bilan personnel 50
8.4 Remerciements 51
9 Bibliographie 52

1 Introduction

Dans le cadre du CFC informatique, j'ai effectué un projet, appelé **Travail pratique individuel (TPI)**, qui a pour but de valider les connaissances que j'ai acquises lors de mon parcours scolaire au CFPT informatique. Ce document accompagne la conception du TPI que j'ai effectué pendant onze jours de travail. Celle-ci était répartie durant les trois premières semaines du mois de mai 2021, du lundi 03 mai au jeudi 20 mai.

FitnesSopt est une application communautaire qui permet à ces utilisateurs de voir des spots de sport sur une map. Chaque spot a sa page d'information, qui contient son adresse, sa note, son nom et des images. Il y a aussi la possibilité d'écrire des commentaires en dessous de chaque spot.

2 Cahier des charges

2.1 Organisation

Élève
Adrian Spycher

Maitre d'apprentissage
Francisco Garcia

Experts	
Serge Murisier	Jean-Pierre Garnier

2.2 Titre

Le titre du projet est **FitnesSpot**.

2.3 Résumé du travail

L'objectif est de créer une application web. C'est une plateforme pour le sport qui offre un répertoire de spot d'entraînement. Une carte permet de trouver facilement les spots de fitness en plein air pour un utilisateur non identifié sur le site. On peut ajouter des spots pour aider les autres à trouver les meilleurs emplacements d'entraînement. On peut devenir un « Spotter » en s'inscrivant sur le site. L'administrateur du site valide les spots insérés par les « Spotter ».

2.4 Matériel et logiciels à disposition

- Un PC standard école, 2 écrans
- Serveur SGBD à choix (EasyPHP, Wamp, Laragon, AdminR, phpMyAdmin, autre)
- Windows 10
- Visual Studio 2017
- Logiciel de création de schémas (Visio, Gliffy, autre)
- Outil de gestion de version de code (Git, avec dépôt distant sur GitHub / Bitbucket / GitLab)

- Navigateur web (Mozilla Firefox/Google Chrome)
- Logiciel de création de maquettes d'interfaces utilisateur (Sketch, Adobe XD, Proto.io, autre)
- Outil bureautique à choix pour les documents (Google Docs, MSOffice, OpenOffice)
- L'étudiant est autorisé à utiliser son matériel personnel au besoin

2.5 Prérequis

Le candidat possède toutes les connaissances nécessaires pour une implémentation professionnelle d'un programme d'un site Web et de gestion des bases de données. Le candidat a suivi les modules ICH-ICT correspondants.

2.6 Descriptif complet du projet

2.6.1 Planification

- Analyse:12h
- Implémentation:36h
- Test:16h
- Documentation:24h

Vous devez établir un planning détaillé selon le format standard.

2.6.2 Méthodologie

Vous devez découper votre travail en tâches et essayer de suivre votre planning durant les 11 jours de réalisation de ce projet. Pour chaque tâche réalisée, pensez à faire des tests avant de la considérer comme terminée, et tester également les autres **FONCTIONNALITÉS** pour éviter les régressions.

2.6.3 Description de l'application

Le site dispose d'une partie (Front-End) :

- Publique
  - Rôle **VISITEUR**
- Utilisateur connecté
  - Rôle **IDENTIFIÉ** ou **SPOTTER**

Le site dispose d'une partie (Back-End) :

- Administration du site
- Rôle **ADMINISTRATEUR**

2.6.4 Prototype d'interface utilisateur

2.6.4.1 Page d'Accueil

Les 3 rôles ont accès à cette page. Il n'est pas nécessaire d'être authentifié sur le site pour découvrir cette page.

Cette page contient:

- Un Banner

- Un logo
- Une barre de recherche qui permet de trouver un ou plusieurs Spot
- Un bouton qui affiche tous les Spots qui redirige vers la page « Spots » décrite plus loin
- Il y a une image miniature du Spot
    - Sous chaque miniature de Spot, on voit le nom de celui-ci, avec le quartier.
    - Le classement sur la page dépend de la popularité, 4 étoiles maximum.
- On peut se connecter ou créer un compte sur le site
- Le **VISITEUR** doit créer un compte pour pouvoir s'identifier
    - Les champs suivants sont obligatoires
        - Pseudo
        - E-Mail
        - Password
- S'identifier
    - Le **VISITEUR** peut s'identifier, seulement s'il est inscrit
    - Un **VISITEUR** authentifié prend le rôle **IDENTIFIÉ**

2.6.4.2 Page Spots

- Les 3 rôles ont accès à cette page. Il n'est pas nécessaire d'être authentifié sur le site pour découvrir cette page.
- La page « Spots » contient la liste de tous les Spots.
- À gauche, il y a une image miniature du Spot.
- À droite de chaque miniature de Spot, on voit le nom de celui-ci, avec le quartier.
- Si on clique sur le quartier, alors la page est rafraîchie avec seulement les Spots du quartier concerné.
- Il est aussi possible de voir uniquement les Spots qui ont 3 étoiles ou plus.
- En bas de page il y a une indication sur la pagination
    - Il y a 6 spots par page.
- Pour « Aimer un Spot », il faut prendre le rôle **IDENTIFIÉ** - **SPOTTER**
- Pour laisser un commentaire, », il faut prendre le rôle **IDENTIFIÉ** - **SPOTTER**
- Avec la loupe, on peut rechercher un spot par son titre ou le quartier.
- On voit une information qui donne le maximum de Spot et où en sommes dans la navigation (1 à 6 de 132 Spots).
- Si on veut voir les détails d'un Spot, il faut cliquer sur la miniature ou le titre.

2.6.4.3 Page Ajouter Spot

Cette page est accessible par le rôle **IDENTIFIÉ** – **SPOTTER** et **ADMINISTATRATEUR**

- On peut ajouter un Spot
- Il faut parcourir la carte et déposer un signet sur la carte.
- On peut modifier la taille de la carte
- On doit au moins « Uploader » une image pour que le Spot soit valide. Il y a quelques formats d'image accepté.
- On peut « Uploader » plusieurs images.

- La taille de l'image est choisie et limitée par vos soins.
- L'information sur le nom du Spot ou la description ne sont pas obligatoire.
- Une fois que le **SPOTTER** a envoyé les informations. **L'ADMINISTATRATEUR** reçoit un message ou un avertissement pour valider le nouveau spot.
- C'est à vous de trouver une méthode d'avertissements et de validation des nouveaux spots.
- Avec le rôle **ADMINISTATRATEUR**, on peut modifier, effacer, consulter, ajouter, valider les spots.
- Avec le rôle **ADMINISTATRATEUR**, on peut modifier, effacer, consulter, ajouter, valider des images à un Spot.

2.6.4.4 Page Carte

Cette page est accessible par les 3 rôles. On y voit une carte qui montre tous les spots. On peut cliquer sur le signet qui nous intéresse et on alors diriger sur la page « un Spot ».

2.6.4.5 Page Un Spot

Cette page est accessible par les 3 rôles

- Le **SPOTTER** peut ajouter un commentaire.
    - **L'ADMINISTATRATEUR** doit valider le commentaire
- **L'ADMINISTATRATEUR** peut aussi enlever, modifier, un commentaire.
- Le **SPOTTER** peut voter de 1 à 4 en cliquant sur le cœur. Il a un seul vote valide, son dernier.
- Le **VISITEUR** ne peut pas ajouter de commentaires. Il ne peut pas voter pour un spot, il doit s'identifier.
- Les 3 rôles voient aussi le nombre d'étoiles reçues pour ce spot.
- Les 3 rôles voient le diaporama et la carte

2.6.4.6 Page Profil

Le **SPOTTER** peut accéder à ces informations uniquement, s'il est connecté - authentifié. Il accède à cette page par Connexion dans le menu de la barre de navigation. Il peut aussi se déconnecter ou se supprimer du site. Par contre, les commentaires ou spots que cet utilisateur a inséré sur le site reste.

2.6.4.7 Page Utilisateurs

Le prototype de cette page fait partie de l'un des points d'évaluation de ce TPI.

- Seul **l'ADMINISTATRATEUR** a accès à cette page.
- **L'ADMINISTATRATEUR** peut voir tous les utilisateurs.
    - Il peut trier par ordre alphabétique ascendant, descendant du Pseudo
- **L'ADMINISTATRATEUR** peut supprimer, modifier, un utilisateur.
- Il peut rechercher une adresse e-mail

2.6.5 MCD

C'est au candidat de créer un MCD valide pour que le site fonctionne correctement. Cela fait partie d'un des critères d'évaluation de ce TPI.

2.7 Livrables

Pour les experts et la formatrice par email :

- Planning détaillé du projet
- Rapport de projet contenant le code source au format PDF
- Journal de bord
- Résumé du TPI (1 page A4)
- Pour le formateur uniquement :
    - L'accès au repository distant du projet avec les droits de « clone »
    - Un readMe explicitant l'installation du projet en local
    - Un dump de la base de données contenant la structure ainsi qu'un set de données de test

2.8 Points techniques évalués spécifiques au projet (obligatoire) correspondants aux points A14 à A20 du formulaire d'évaluation

A14: Le MCD est correct et correspond au besoin du fonctionnement du site.

A15: **L'ADMINISTRATEUR** est averti qu'un nouveau spot ou qu'un nouveau commentaire a été ajouté par un **IDENTIFIÉ - SPOTTER** pour validation

A16: Le rôle **VISITEUR** ou **IDENTIFIE** peuvent voir la liste des spots disponibles et peuvent les filtrer

A17 : Le rôle **VISITEUR** ou **IDENTIFIE** peuvent arriver sur le Spot choisi par le biais du signet depuis la page Carte

A18: Le rôle **IDENTIFIÉ** ajouter un spot

A19: Le rôle **IDENTIFIE** peut voter pour un SpotA20: Le rôle **ADMINISTRATEUR** peut gérer les utilisateurs comme décrit.

3 Méthodologie

Pour pouvoir organiser mon travail, j'ai choisi d'utiliser la méthode en 6 étapes que j'ai mises en pratique lors de la majorité de mes projets, entre autres pendant le module M306. Cette méthode nous permet de séparer la conception du projet en plusieurs étapes, comme montrer dans le diagramme ci-dessous.

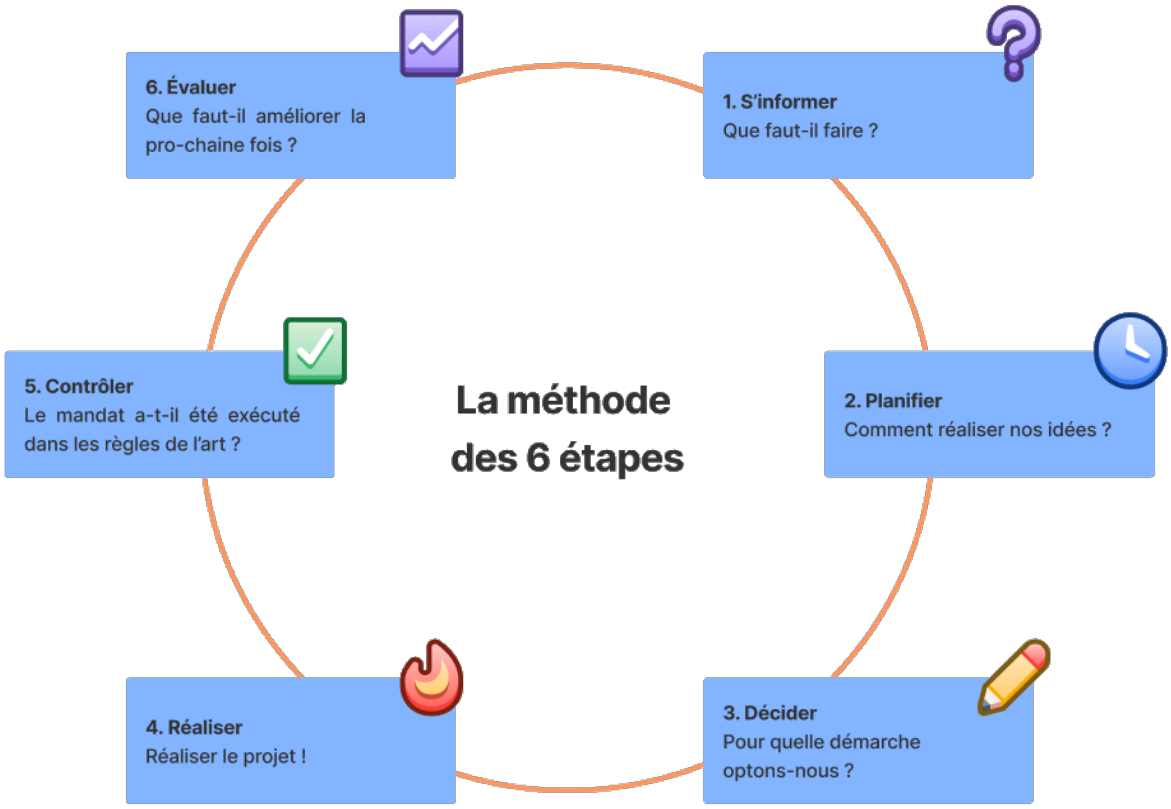


Figure 1 - Diagramme de la méthode des 6 étapes

3.1 S'informer

La première étape consiste à cerner le sujet de façon à pouvoir comprendre correctement ce que l'on nous demande. Il s'agit aussi de se créer une idée de l'objectif final. De plus, c'est l'étape dans laquelle nous allons essayer d'obtenir un maximum d'information du mandant.

Dans mon cas, les informations nécessaires étaient déjà toutes fournies dans le cahier des charges, j'ai juste eu à demander quelques clarifications.

3.2 Planifier

La planification est la deuxième étape, elle consiste à planifier toute l'organisation du travail pour arriver au but. Il s'agit de mettre en ordre les différentes opérations en prenant compte du niveau des priorités de chacune. Et d'ensuite estimer le temps nécessaire qu'il faut prendre pour effectuer la tâche. Dans le cadre d'un projet individuel, il n'est pas nécessaire de

départager les tâches entre les collaborateurs, mais c'est une étape qu'il faut prendre en compte dans le cas contraire.

Pour planifier la réalisation de mon projet, j'ai organisé un *Product Backlog* avec toutes les *User Story* qui m'était demandé dans le cahier des charges. Cela m'a aidé à la création d'un diagramme de Gantt, qui m'a servi de repère vis-à-vis de l'avancement de mon travail. La gestion de l'importance des tâches m'a permis de vite me retrouver avec un *minimum viable product* (le produit minimum viable ou MVP).

3.3 Décider

La troisième étape est le moment durant lequel nous allons faire des choix. Il peut par exemple s'agir de trouver des normes que les collaborateurs devront respecter, la façon dont certains fichiers seront traités ou encore les couts que les tâches peuvent avoir. C'est aussi pendant cette étape que nous allons choisir les outils et les moyens auxiliaires pour réaliser notre travail.

Dans mon cas, j'ai étudié la question de l'implémentation de la map sur quelle technologie j'allais me diriger (ex : Google Map, OSM, Mapbox). Pour ce qui était de la façon dont j'allais écrire les documents, je me suis fié à des modèles que l'on m'avait donnés et que j'avais traités au préalable.

3.4 Réaliser

La quatrième étape est la réalisation. Elle consiste à exécuter les tâches que nous avons planifiées et décider au préalable. C'est en général l'étape la plus longue du projet, il faut se fier un maximum au plan établi.

Lors de la réalisation, j'ai tenu un journal de bord qui m'a servi à expliquer le fond de ma pensée de façon quotidienne. C'est aussi durant la réalisation que j'ai effectué le plus gros du travail. Pour ma part, la réalisation s'est bien passée, entre autres, car j'ai bien planifié le déroulement du projet.

3.5 Contrôler

Les contrôles se font lors de la cinquième étape. À ce moment-là, chaque *User Story* doit être complet, pour qu'elle soit ensuite vérifiée. Par exemple, nous pouvons vérifier si la description et le résultat de la tâche correspondent bien aux consignes établies. Il s'agit aussi d'effectuer un plan de test sur le projet en lui-même. Il est conseillé de faire effectuer les contrôles et les tests à des personnes tierces, car cela apporte de meilleurs résultats.

Pour ma part, j'ai choisi de faire des tests quotidiens. C'est, entre autres, pour voir plus simplement l'avancement de mon travail. J'avais aussi pensé à le faire uniquement à la fin de la conception de mon projet, mais je n'ai pas opté pour cette façon de faire, car je trouvais que cette méthode néglige trop les tests intermédiaires.

3.6 Évaluer

La sixième et dernière étape est l'évaluation du travail. Lors de l'évaluation, nous passons en revue le déroulement du projet et nous repérons les aspects qui ont bien fonctionné et ceux qui ont moins bien fonctionné. De plus, pour ne pas avoir à refaire les mêmes erreurs, nous cherchons à connaitre les points qui pourraient être améliorés pour le prochain projet.

Dans mon cas, j'ai fait une rétrospective de mon projet le dernier jour, et j'écris toutes critiques que j'ai faites vis-à-vis du déroulement de mon projet dans la conclusion. De plus, j'y écris un bilan personnel du ressenti que j'ai eu à faire ce dernier projet.

4 Planification

4.1 Product Backlog

Avant la création d'un planning, j'ai organisé un Product Backlog qui me sert de référence pour toute la durée du projet. Le Product Backlog est une liste contenant toutes les User Story du projet. Elles sont séparées en quatre parties. Le titre qui représente en gros l'User Story. La description qui décrit l'User Story en détail. Le test associé qui désigne les tests qui y font référence, il peut y en avoir plusieurs. Ainsi que l'importance qui désigne l'importance de la tâche sur le projet. Elles sont toutes numérotées, d'un à dix-huit :

Titre	**1 – Récupération des données**
Description	Récupération des différents spots de sport existant ainsi que des images libres de droits. Utilisation d'overpass-turbo pour les spots et Unsplash pour les images.
Test associé	Le test 1
Importance	⚠⚠

Titre	**2 – Élaboration de la base de données**
Description	Création de la base de données de manière à ce qu'elle convienne parfaitement à l'usage du site. Ainsi que l'insertion des fausses données dedans.
Test associé	Le test 2
Importance	⚠⚠⚠

Titre	**3 – Implémentation de la map**
Description	Implémentation de la map dans la page carte et dans le formulaire d'ajout de spot. La map peut s'agrandir dans la page d'ajout. Elles doivent être responsive.
Test associé	Les tests 3.1 et 3.2
Importance	⚠⚠

Titre	4 – Affichage des spots populaires
Description	Affichage des spots populaires dans la page d'accueil, il doit y contenir les informations suivantes : image, nom, adresse et popularité.
Test associé	Le test 4
Importance	⚠⚠

Titre	5 – Recherche d'un spot
Description	Barre de recherche qui nous permet de trouver un spot rapidement et simplement, par rapport au nom ou à l'adresse d'un spot. Lorsque je valide ma recherche, l'affichage est trié par ordre de pertinence.
Test associé	Les tests 5.1 et 5.2
Importance	⚠⚠⚠

Titre	6 – Affichage des spots sous forme de liste
Description	Affichages des spots sous forme de liste. Il doit contenir les informations suivantes : images, possibilité d'aimer le spot, nom, l'adresse, et la popularité. Il y a une pagination qui comporte six spots par page. Cliquer sur le spot nous ramène sur sa page.
Test associé	Les tests 6.1, 6.2, 6.3 et 6.4
Importance	⚠⚠⚠

Titre	7 – Filtrage des spots
Description	Dans la page spots, on peut filtrer les spots par popularité. Et par adresse si on clique sur l'adresse d'un des spots.
Test associé	Les tests 7.1 et 7.2
Importance	⚠⚠

Titre	8 – Gestion de popularité
Description	On peut aimer un spot en cliquant sur un cœur. Il se trouve dans la page spot, et dans la page d'information d'un spot. On peut aussi voter pour un spot avec un nombre d'étoiles, cela se trouve dans la page d'information du spot.
Test associé	Les tests 8.1 et 8.2
Importance	⚠⚠

Titre	9 – Ajout d'un spot
Description	Sur la page d'ajout de spot, on peut ajouter un spot en rentrant sa position à l'aide d'une map, son nom, sa description et une image du spot. Le nom et la description du spot ne sont pas obligatoires. Si un post est ajouté, cela doit notifier l'administrateur pour qu'il le valide.
Test associé	Les tests 9.1, 9.2 et 9.3
Importance	⚠⚠

Titre	10 – Affichage des spots sous forme de marqueur.
Description	Sur la page carte, on doit pouvoir voir tous les spots sur une carte. Si on clique sur le post, une pop-up apparait avec son nom. En cliquant sur le nom, on est redirigé sur sa page d'information.
Test associé	Le test 10
Importance	⚠⚠

Titre	11 – Affichage des informations d'un spot
Description	Sur la page d'information d'un spot, on peut voir toutes les informations d'un spot. C'est-à-dire, son nom, son adresse, sa popularité, les commentaires ainsi que ses images.
Test associé	Le test 11
Importance	⚠⚠

Titre	12 – Gestions des spots
Description	Sur la page d'information d'un spot, l'administrateur doit pouvoir modifier ou supprimer un spot. Si le spot n'est pas validé, il peut le valider.
Test associé	Le test 12
Importance	⚠⚠

Titre	13 – Gestions des commentaires
Description	Sur la page d'information d'un spot, on peut ajouter des commentaires, uniquement si on est connecté. L'administrateur doit pouvoir modifier, supprimer et valider un commentaire.
Test associé	Le test 13
Importance	⚠⚠

Titre	14 – Gestion de son profil
Description	Chaque utilisateur connecté a accès à la page de son compte. Il peut modifier ces informations. Il peut se déconnecter via cette page et aussi se supprimer du site.
Test associé	Les tests 14.1, 14.2 et 14.3
Importance	⚠⚠⚠

Titre	15 – Gestion des utilisateurs
Description	L'administrateur a accès à une page gestion des utilisateurs. Il peut voir tous les utilisateurs et les trier par ordre ascendant ou descendant de leurs pseudos. Il peut supprimer ou modifier un utilisateur. Il peut rechercher par adresse email.
Test associé	Les tests 15.1 et 15.2
Importance	⚠⚠

Titre	16 – Gestion des rôles
Description	Les utilisateurs ont de base le rôle visiteur. S'il se connecte, ils obtiennent le rôle d'identifier. Certains compte son marqué en tant qu'administrateurs. Chaque rôle est respecté, un utilisateur ne peut pas effectuer des tâches d'un identifié, etc.
Test associé	Les tests 16.1, 16.2 et 16.3
Importance	⚠⚠⚠

Titre	17 – Création d'un compte
Description	Un utilisateur doit pouvoir créer un compte via un formulaire d'inscription.
Test associé	Les tests 17.1 et 17.2
Importance	⚠⚠⚠

Titre	18 – Connexion au site
Description	Un utilisateur doit pouvoir se connecter au site, s'il a un compte.
Test associé	Les tests 18.1 et 18.2
Importance	⚠⚠⚠

4.2 Planning

Pour planifier mon projet, j'ai utilisé la méthode de Gantt. C'est un système souvent utilisé dans la gestion de projet. Pour ce faire, j'ai séparé en six parties les tâches que j'avais à faire. Ces parties reprennent la méthode en six étapes. Pour ce qui est de la réalisation, je me suis référé au Product Backlog. J'ai organisé les User Story du Product Backlog dans le diagramme en les séparant par page.

Pour la temporalité du diagramme, la méthode de Gantt prévoit de la mettre au-dessus des tâches, de façon à pouvoir cocher quelle tâche est faite à quelle période. Dans mon projet, je ne pouvais pas travailler en dehors des cours, alors j'ai dû enlever les week-ends, le vendredi qui est réservé pour la maturité professionnelle et j'ai limité les jours à huit heures.

La méthode de Gantt est très efficace grâce aux visuelles qu'elle revoie. Elle permet de voir très simplement :

- L'avancement des différentes User Story
- La date de début et de fin de chaque User Story
- La durée qu'une tâche prend à être effectuée
- La priorité de chaque tâche
- La date de début et de fin du projet

Le planning que j'ai inséré dans la documentation n'est pas l'entièreté du planning. C'est volontaire pour laisser de la place à la réalisation qui est la partie la plus importante du planning. L'intégralité du planning se trouvera par conséquent en annexe. Le planning que j'ai créé est le suivant :

Listes des tâches	Semaine 19 lun 3 mai 2021	Semaine 19 mar 4 mai 2021	Semaine 19 mer 5 mai 2021	Semaine 19 jeu 6 mai 2021	Semaine 20 lun 10 mai 2021
2 - Planification					
Création du Journal de bord.	p p p p p p p p p p p p p p p p p p p p				
Création de la documentation.	p p p p p p p p p p p p p p p p p p p p				
4 - Réalisation					
1 - Récupération des données			p p		
2 - Élaboration de la base de données			p p		
4 - Affichage des spots populaires			p p p	p p	
5 - Recherche d'un spot				p p p p p	
6 - Affichage des spots sous forme de liste				p	p
7 - Filtrage des spots					p p p
8 - Gestion de la popularité					p p p
9 - Ajout d'un spot					
3 - Implémentation de la map					
10 - Affichage des spots sous forme de marker					
11 - Affichage des informations d'un spot					
12 - Gestions des spots					
13 - Gestions des commentaires					
14 - Gestion de son profil					
15 - Gestion des utilisateurs					
16 - Gestion des rôles					
17 - Création d'un compte					
18 - Connexion au site					
Création du manuel utilisateur.					
5 - Contrôles					
Effectuer le plan de teste.	p p p p p p p p p p p p p p p p p p p p				

Listes des tâches	Semaine 20 mar 11 mai 2021	Semaine 20 mer 12 mai 2021	Semaine 21 lun 17 mai 2021	Semaine 21 mar 18 mai 2021	Semaine 21 mer 19 mai 2021	Semaine 21 jeu 20 mai 2021
2 - Planification						
Création du Journal de bord.	p p p p p p p p p p p p p p p p p p p p					
Création de la documentation.	p p p p p p p p p p p p p p p p p p p p					
4 - Réalisation						
1 - Récupération des données						
2 - Élaboration de la base de données						
4 - Affichage des spots populaires						
5 - Recherche d'un spot						
6 - Affichage des spots sous forme de liste						
7 - Filtrage des spots						
8 - Gestion de la popularité						
9 - Ajout d'un spot	p p p					
3 - Implémentation de la map	p p					
10 - Affichage des spots sous forme de marker	p p					
11 - Affichage des informations d'un spot		p p				
12 - Gestions des spots		p p p p				
13 - Gestions des commentaires		p	p p			
14 - Gestion de son profil			p p			
15 - Gestion des utilisateurs			p p			
16 - Gestion des rôles			p	p p		
17 - Création d'un compte				p p		
18 - Connexion au site				p p		
Création du manuel utilisateur.					p p p p	
5 - Contrôles						
Effectuer le plan de teste.	p p p p p p p p p p p p p p p p p p p p					

5 Généralités concernant l'implémentation

5.1 Librairies et outils externes utilisés



Figure 2 - Icône de l'IDE Visual Studio Code

Visual Studio Code ou souvent appeler **VsCode** est un environnement de développement (IDE) développé par *Microsoft*. C'est selon *Stack Overflow* l'outil de développement le plus populaire, et cela s'explique surement dû à ces nombrables extensions qui permette d'utiliser cet outil dans la plupart des cas imaginables.

Pour ma part, j'ai utilisé Visual Studio Code pour quelque des ces extensions. GitLens qui permet d'intégrer Git à l'IDE. Cela simplifie l'utilisation de git, notamment dans les *push* et les *commit* et il nous offre un affichage très intuitif de notre projet. VsCode nous propose aussi des extensions pour l'utilisation d'un débogueur en PHP, ou bien de formateur de code, que j'ai évidemment utilisé.



Figure 3 - Icône de la plateforme de développement web Laragon

Laragon est une plateforme de développement Web, permettant de faire fonctionner localement des scripts *PHP*. Il utilise un *WAMP* serveur qui est un environnement comprenant trois serveurs *Apache*, *MySQL* et *PHP*. Il est simple d'utilisation et permet d'installer certains outils comme *PhpMyAdmin* simplement.

Pour ma part, j'ai utilisé ce logiciel pour pouvoir simplement créer un environnement de développement local. J'ai hésité à créer un serveur avec la sorte de machine virtuelle de

Debian que propose *Windows*, mais l'ayant utilisé trop peu de fois et ayant eu quelques problèmes de droits, j'ai préféré rester sur l'utilisation d'un outil fiable.



Figure 4 - Icône du logiciel d'administration de base de donnée DBeaver

Dbeaver est un logiciel client SQL qui offre un outil d'administration de bases de données. Il fournit un éditeur qui prend en charge la complétion de code et la coloration syntaxique. Son interface graphique permet de facilement effectuer des tâches qui peuvent sembler compliquées. Il offre également la possibilité d'afficher sous forme de diagramme une base de données.

Pour ma part, ce logiciel est à la base de toutes la conception de ma base de données. Il m'a permis de la créer et de la modifier sans crainte. De plus, j'ai pu utiliser son éditeur de script *SQL* afin de pouvoir créer et vérifier facilement mes requêtes avant de les implémenter dans mon code.



Figure 5 - Icône de l'outil de gestion de version GitHub

Git est un outil de gestion de version décentralisé, gratuit et open source. Il est simple à apprendre et permet de gérer du plus petit projet au plus grand. Pour exploiter cet outil, j'ai utilisé l'intermédiaire **GitHub** qui est un service web d'hébergement de projet. De plus, en tant qu'élève, je profite de l'intégralité des fonctionnalités qu'il propose.

Pour ma part, j'ai utilisé ces outils dans le but de pouvoir sauvegarder mes projets à un autre endroit que sur mon disque dur externe. De plus il m'a permis de faire des versions sur les étapes de mon projet. J'ai créé deux dépôts Git, l'un pour sauvegarder mon projet et l'autre pour enregistrer ma documentation.



Figure 6 - Icône du service de cartographie Mapbox

Mapbox est un service de cartographie, autrement dit une *API*, qui propose énormément d'outils. Évidement il propose un affichage de map sur site web, mais aussi d'autre service tels que, des systèmes de chercher d'adresse, de navigation GPS avec des itinéraires ainsi que la possibilité d'intégrer ces services dans des applications mobiles.

Dans mon projet, j'ai principalement utilisé les services d'affichage de map et *l'api* de recherche d'adresse. J'aurais pu utiliser d'autres outils de cartographie comme *Google Map* ou *OpenStreetMap* mais j'ai opté pour celui-ci, car j'avais déjà mis en place un site web utilisant ces services. Mapbox a comme plus-value de rendre son utilisation très simple et intuitive. De plus, c'est une bonne alternative au niveau de son prix, car il est entièrement gratuit pour un usage simple, comme mon projet.

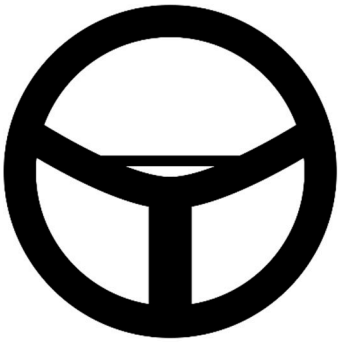


Figure 7 - Logo du service de base de données Overpass-Turbo

Overpass API est un service qui permet d'interroger la base de données d'*OpenStreetMap*. Elle offre la possibilité de créer des requêtes très complètes qui vont permettre à l'utilisateur de récupérer des informations selon plusieurs critères tels que des tags des objets, types d'objets, localisation géographique, etc. **Overpass-Turbo** est un site web qui propose une interface pour faciliter l'utilisation de *Overpass API*. Elle donne un assistant qui permet de générer des requêtes très simplement.

Dans mon cas, j'ai utilisé ces deux services pour pouvoir importer des données concernant des parcs de musculations, dans ma base de données. Ce n'était pas quelque chose dont j'étais obligé de faire, mais je tenais à pouvoir essayer mon site avec un grand nombre de données. Et vous vous en doutez, je n'allais pas taper ces données à la main.



Figure 8 - Icône du logiciel de test d'API Postman

Postman est un logiciel de test d'*API*. Il consiste à tester des *API* afin de déterminer si elles répondent aux attentes demandées, en matière de fonctionnalité, de fiabilité, de performance et de sécurité. Il crée une interface simple d'utilisation entre une *API* quelconque et l'utilisateur de cette dernière.

Je l'ai utilisé pour tester les appelle à l'*API* Mapbox que je faisais. Il m'a été très utile pour être sure que mes appels étaient fonctionnels et le plus optimisé possible. Je pouvais les essayer directement dans le code, mais celui-ci ne m'aurait donné aucune erreur concrète, et du débogage long et inutile.



Figure 9 - Icône de la nouvelle approche web AJAX

AJAX (Asynchronous JavaScript + XML) est une nouvelle approche utilisant une bonne partie des technologies qui existe dans le monde du web, entre autres l'HTML, le CSS et le JavaScript. Grâce à l'AJAX, les applications web sont capables de rafraichir leurs contenus sans avoir à rafraichir la page, de façon asynchrone.

Dans mon projet, j'ai utilisé l'AJAX pour pouvoir modifier le contenu de ma page sans que la page soit rechargée. C'est lorsque l'utilisateur fait des petites interactions qui ne nécessitent pas un rechargement complet de la page que j'utilise.



Figure 10 - Icône du Framework CSS Bootstrap 5

Bootstrap 5 est un Framework CSS qui permet de concevoir et personnalisez très rapidement des sites web, en prenant compte de la taille de l'écran utilisé. C'est l'un des Framework CSS les plus utilisés au monde. Il est très complet et permet l'utilisation de variable, de système de grille responsive, ainsi que de nombreux composants préconstruits. Par conséquent, certaines de ces fonctionnalités utilisent de puissants plug-ins JavaScript.

Pour ma part, j'ai utilisé ce Framework CSS, car il m'a permis de très bien styliser mon site web sans me poser trop de questions. J'ai choisi ce Framework CSS, car il avait déjà fait ses preuves dans d'autres de mes projets, ayant la main dessus, je tenais à le réutiliser.

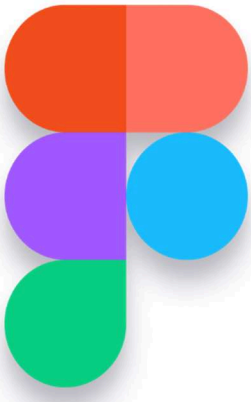


Figure 11 - Icône de l'outil graphique Figma

Figma est un outil graphique. Il permet de créer des prototypes de façons vectoriels. Il est principalement utilisé sur navigateur internet, mais peut aussi être installé en tant que logiciel. Ce qui le diffère de ces concurrents, c'est qu'il est entièrement gratuit pour une utilisation individuelle. Car oui, il offre la possibilité de travailler à plusieurs sur un même projet.

Pour ma part, je l'ai utilisé pour créer une maquette, pas très détaillée, de certaines pages du site. Cela me permettait de me faire une idée de la page que j'allais créer avant de me lancer dans le code ce qui m'évitait des erreurs bêtes, ou de devoir trop réfléchir pendant la conception du site. De plus j'ai pu profiter de leur nouveau système de diagramme qui m'a permis de schématiser certaines fonctionnalités.

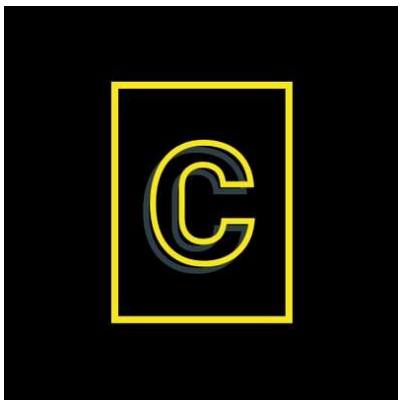


Figure 12 - Icône de l'outil d'insertion de code source Carbon

Carbon est un site web permettant de créer et partager des images contenant du code source. Il permet d'intégrer plus facilement du code dans des fichiers de bureautique tels que *Word*. De plus, les différents types affichages qu'il propose permettent au code de très bien s'intégrer dans des fichiers qui sont de base remplie de texte.

J'ai utilisé Carbon tout du long de ma documentation, pour pouvoir afficher des morceaux de code source que je devais expliquer. J'ai trouvé cela bien plus propre et soigné que de faire une simple capture d'écran. Je n'ai toutefois pas utilisé ce site dans mon journal de bord, car le Markdown permet déjà une implémentation de code source.



Figure 13 - Icône de la bibliothèque d'image Unsplash

Unsplash est ma principale source d'images libres de droits. C'est un site web qui est dédié au partage de photos sous licence libre. Il propose plus de 2 millions de photos haute définition libres de droits. Elles sont téléchargeables sans contrainte et peuvent être utilisées à des buts commerciaux ou non.

Dans mon projet, c'est ce site web qui m'a fourni les images nécessaires pour compléter le design du site. Cela va aux images de fond principal de mon site web, jusqu'aux fausses données que j'ai insérées par la suite de mon projet.

5.2 Architecture du projet

5.2.1 Modèle Vue Contrôleur

L'architecture de mon projet se base sur le motif Modèle-vue-contrôleur ou MVC. Comme on peut s'y attendre, ce type d'architecture sépare le projet en trois parties distinctes, la première étant le modèle, la seconde la vue et la dernière le contrôleur. Chaque de ces parties à sa propre fonction.

- Le modèle contient ou récupère les données à afficher.
- La vue contient la partie graphique du projet, autrement dit l'affichage.
- Le contrôleur contient la logique concernant les actions, c'est lui qui va filtrer les données et les analyser.

Voici une image qui schématise bien l'architecture MVC :

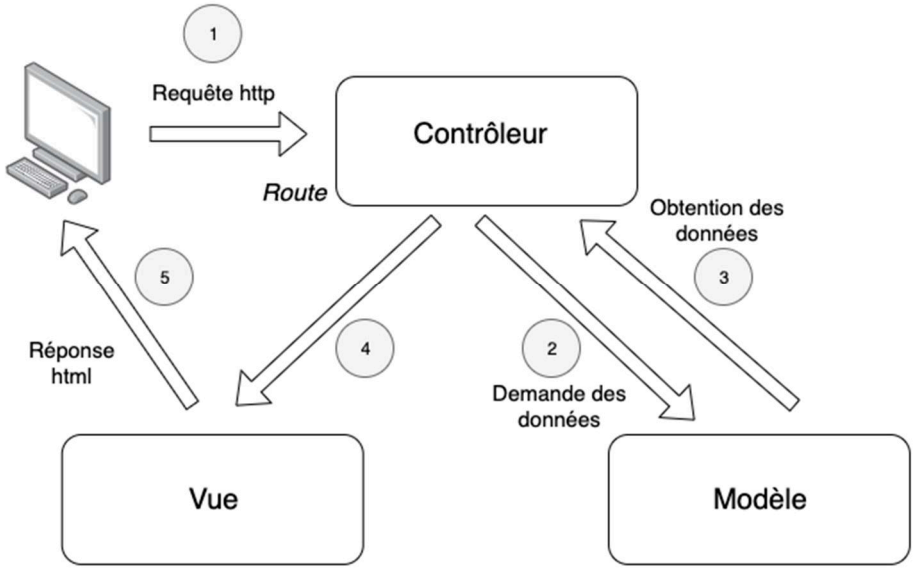


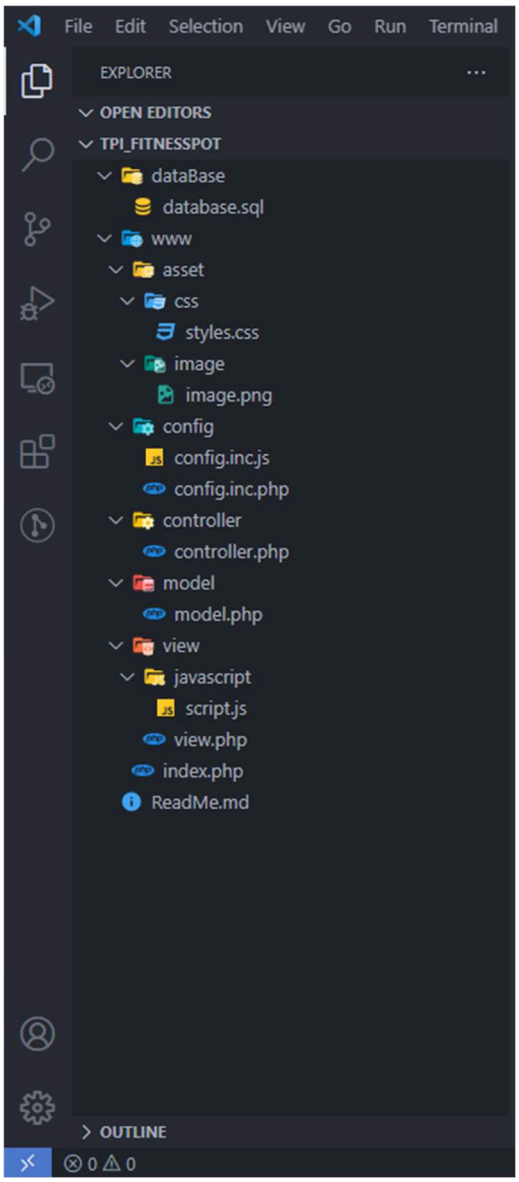
Figure 14 - Interactions entre le modèle, la vue et le contrôleur.

Si j'ai choisi cette façon de procéder, c'est parce que ce parton de conception est très adapté aux applications web. Le cycle action - mise à jour - affichage que l'on retrouve dans ce patron organise parfaitement les différentes étapes qui sont demandées lors de l'action d'un utilisateur. De plus, l'architecture MVC permet de séparer les couches, ce qui améliore considérablement la maintenabilité de l'application.

5.2.2 Arborescence

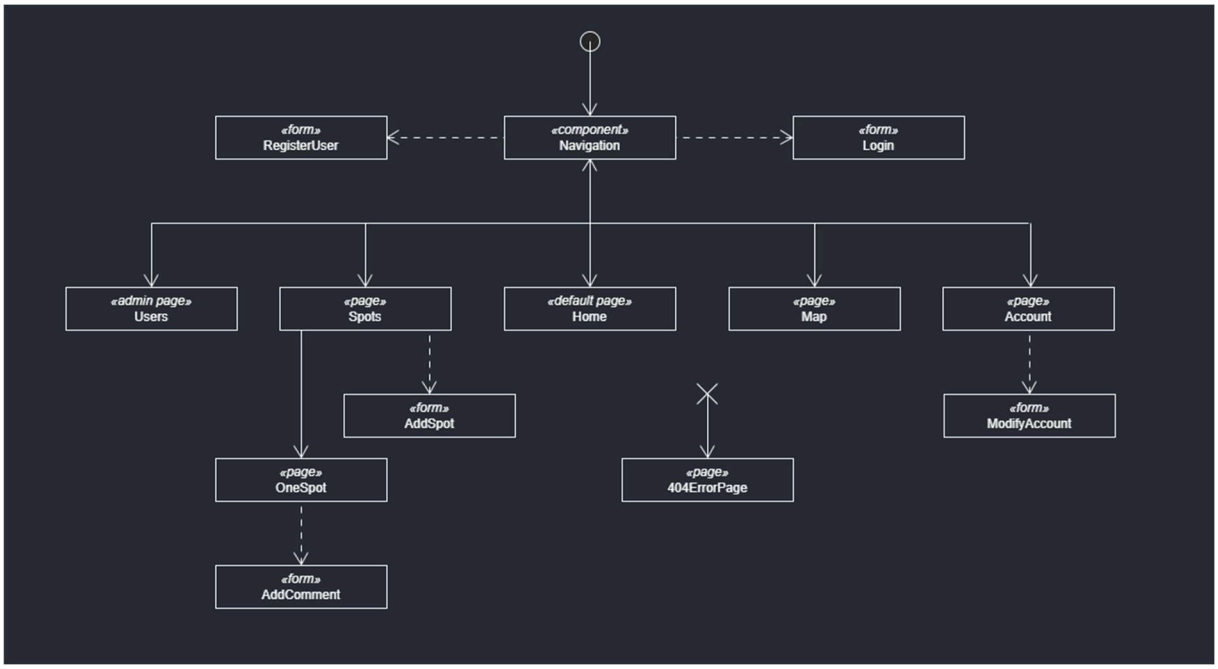
L'arborescence de mon projet s'organise autour de plusieurs répertoires importants :

- **dataBase** : contiens la structure de la base de données avec un utilisateur ayant les droits dessus ainsi que des fichiers contenant quelques données de base.
- **www** : est la racine de mon site web, il renferme tous les sous-répertoires qui servent au bon fonctionnement du site web.
- **asset** : contiens toutes les ressources du projet.
    - **css** : comporte les fichiers de style du site web.
    - **image** : inclus l'entièreté des images de l'application web.
- **config** : contiens les fichiers de configuration des services externes et de la base de données.
- **controller** : englobe tous les contrôleurs du projet qui détiennent la logique du projet.
- **model** : renferme les différents modèles du projet, ils disposent des données du projet.
- **view** : contiens les pages du projet, c'est lui qui fait le lien entre l'utilisateur et le site.
    - **javascript** : inclus tous les scripts JS du projet, c'est eux qui dynamisent le site.



5.2.3 Diagramme de navigation

Avant de passer à la conception de mon projet, j'ai fait un diagramme de navigation. Il prend en compte toutes les pages. Je n'ai, par contre, qu'afficher les principales redirections, et pas toutes les redirections secondaires que l'on peut trouver sur le site, par exemple si l'on clique sur l'image d'un spot.



5.3 Base de données

Le diagramme de base de données ne m'a pas été fourni par mon maitre d'apprentissage. J'ai alors construit une base de données de rien. J'ai choisi une base de données MySQL pour stocker mes informations. Elle est encodée en utf8 avec comme collation par défaut utf8_general_ci et utilise un moteur InnoDB.

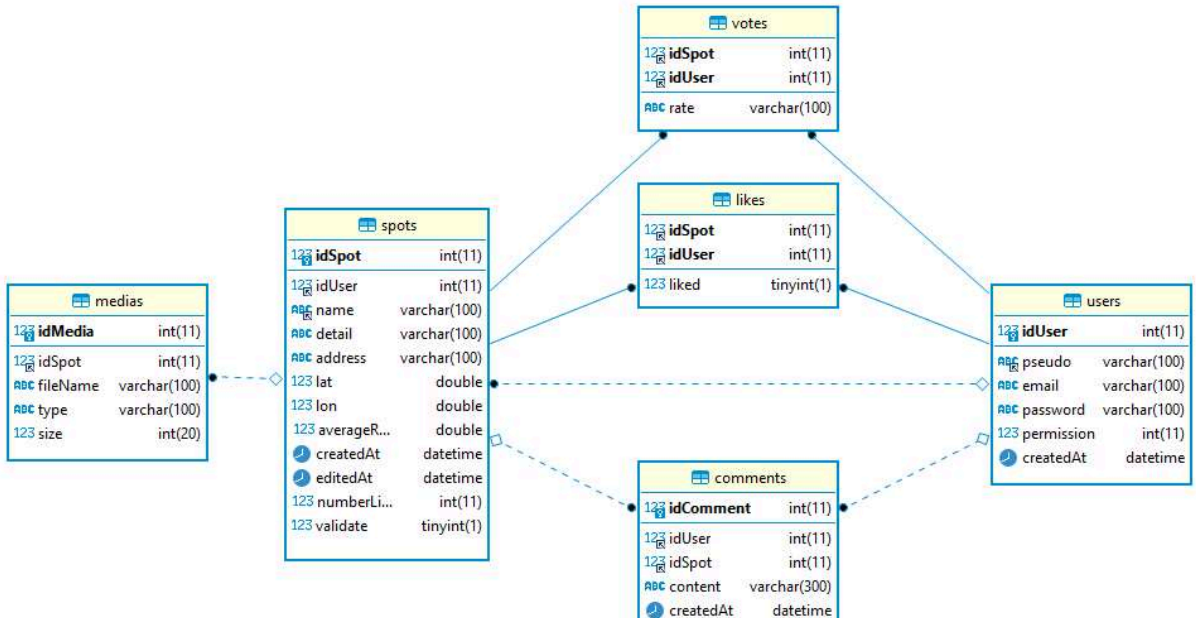


Figure 15 - Modèle conceptuel de la base de données de FitnesSpot

5.4 Fonctionnalités intégrées

Les fonctionnalités que j'ai implémentées dans mon projet sont décrites ci-dessous. Je les ai séparés par rapport aux pages sur lesquels elle agisse.

Invité - La page d'accueil :

- Affichage des spots les plus populaires
- Insertion d'une barre de recherche, celle-ci redirige à la page spots
- Implémentation d'un bouton qui redirige sur la page de tous les spots.

Invité - La page de tous les spots :

- Affichage sous forme de liste tous les spots du site web
- Implémentation d'une pagination de manière à ne pas afficher tous les spots d'un coup, mais à coup de six spots par page
- Élaboration d'une barre de recherche qui filtre les spots
- Création d'un bouton qui permet d'afficher uniquement les spots de plus de trois étoiles

Invité - La page carte :

- Implémentation d'une map, permettant l'affichage de tous les spots
    - Contraints de ne pas afficher tous les spots d'un coup sur la map, sous risque de surcharger le site
- Implémentation de pop-up qui permette d'identifier chaque spot, et de se rediriger sur la page d'information du spot

Invité - La page d'information d'un spot :

- Affichage des différentes données du spot
- Créer la possibilité de donner une note au spot
- Donner la possibilité d'aimer le spot
- Implémentation d'une rubrique de commentaire, et d'un formulaire d'ajout de commentaire

Utilisateur - La page d'ajout d'un spot :

- Élaboration d'un formulaire permettant d'ajouter un spot
    - Implémentation d'une map permettant la localisation du spot
- Envoie d'un mail à l'administrateur, en cas d'ajout de spot

Utilisateur - La page profile :

- Affichage des données de l'utilisateur
- Possibilité de supprimer son compte
    - En supprimant son compte, il ne supprime pas ces votes, ces « j'aime » et ces commentaires

Admin - La page utilisateur :

- Affichage de tous les utilisateurs sous forme de liste
- Implémentation d'une pagination de manière à ne pas afficher tous les utilisateurs d'un coup
- Élaboration d'une barre de recherche qui filtre les utilisateurs

Autre :

- Élaboration du modèle conceptuel de donnée et de la base de données
- Récupération ou création d'un nombre important de données permettant l'utilisation et le test du projet
- Formulaire de création d'un compte
- Formulaire de connexion à un compte
- Gestion des rôles et de l'accessibilité aux pages

6 Analyse des fonctionnalités majeures

6.1 Singleton de connexion à la base de données

La classe *DBConnection* sert à établir une unique instance de connexion à la base de données, en utilisant l'objet PDO de PHP. Ceci est fait grâce au patron de conception **Singleton** dont le principe est de créer la connexion à la base de données qu'une seule fois. Pour interdire à l'objet sa création et son clonage, nous avons mis ses fonctions *__construct* et *__clone* en privé. Pour rendre la connexion unique, nous avons utilisé une variable statique qui stocke la connexion à la base de données. Pour forcer cette variable statique à être unique, nous l'avons mis en privée et nous avons créé une fonction *getConnection* qui vérifie si la connexion à déjà été faite ou non. Si la connexion est déjà faite, il lui suffit de la retourner. Dans le cas contraire, il en créera une grâce à la fonction *doConnection*.

```php
class DBConnection
{
    private static $conn = null;    // conn to the database

    // [...]
    private function __construct() { }

    // [...]
    private function __clone() { }

    /**
     * @brief   Create initial connection to a DB
     * @return  result (boolean);
     */
    private static function doConnection()
    {
        $result = true;

        try {
            // create the connexion
            self::$conn = new PDO("mysql:host=HOST;dbname=DB_NAME;charset=utf8", DB_USER, DB_PWD);
            self::$conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch (PDOException $e) {
            // return false and error
            $result = false;
            echo '<pre>Erreur : ' . $e->getMessage() . '</pre>';
            die('Could not connect to MySQL');
        }

        return $result;
    }

    /**
     * @brief   Create a static conn, or return one already created.
     * @return  conn (obj);
     */
    public static function getConnection()
    {
        if (self::$conn == null)
            self::doConnection();

        return self::$conn;
    }

    // [...]
}
```

La méthode *__callStatic* est, en réalité, une méthode qui est appelée lorsque l'on invoque des méthodes inaccessibles ou qui n'existe pas, dans un contexte statique. Le paramètre *$chrMethod* est le nom de la méthode appelée et le paramètre *$arrArguments* est un tableau contenant les paramètres passés à la méthode *$chrMethod*.

Tout d'abord on remarque que la fonction va chercher à récupérer l'objet de connexion PDO. Par la suite, la fonction PHP *call_user_func_array* nous permet d'appeler une fonction par rapport aux paramètres qui lui sont passés. Dans cet exemple, *call_user_func_array* nous permettra appeler une quelconque fonction de l'objet PDO, en se basant sur ces paramètres. Autrement dit, cela nous permet, via la classe *DBConnection*, d'utiliser n'importe quelles fonctions de l'objet PDO.

```php
class DBConnection
{
    // [...]

    /**
     * @brief   Passes on any static calls to this class onto the singleton PDO instance
     * @param   chrMethod       The method to call
     * @param   arrArguments    The method's parameters
     * @return  mix             The method's return value
     */
    final public static function __callStatic($chrMethod, $arrArguments)
    {
        $objInstance = self::getConnection();
        return call_user_func_array(array($objInstance, $chrMethod), $arrArguments);
    }
}
```

Il faut préciser que nous avons reçu cette classe et appris cette façon de faire lors d'une cour d'atelier support qui nous est présenté par notre professeur, Monsieur Aigroz.

6.2 Mes modèles, les Prepared Statements et les Bound Parameters

Lors de la conception des modèles présents dans mon projet, j'ai voulu mettre à bien une procédure que nous avions vue dans de précédents cours. Il s'agit de l'utilisation des Prepared Statements. Dans cette façon de faire, le principe est d'ajouter une étape supplémentaire à l'exécution d'une requête SQL, pour pouvoir l'optimiser. Cette étape est la préparation de la requête. Le nombre d'étapes s'élève alors à deux : la préparation et l'exécution. Au niveau serveur, la préparation se fait en trois étapes :

Premièrement, le modèle d'une requête SQL sera créé et envoyé à la base de données. Les valeurs qui nécessitent un changement plus tard, sont remplacées par des "*?*" ou bien des paramètres tels que : *:idUser*.

Deuxièmement, la base de données analysera, compilera et optimisera les différentes requêtes possibles depuis le modèle SQL envoyé. Il stockera les résultats sans pour autant avoir exécuté la requête.

Troisièmement, plus tard, lors de l'exécution de la requête, l'application liera les valeurs aux paramètres du modèle envoyé préalablement, et exécutera la requête. L'application peut exécuter l'instruction autant de fois qu'elle le souhaite avec des valeurs différentes.

Les principales raisons pour lesquels j'ai utilisé des Prepared Statements est qu'ils optimiser grandement les performances d'utilisation. Et cela de plusieurs façons :

Tout d'abord, le temps d'analyse de la requête est drastiquement réduit, car l'analyse se fait qu'une seule fois.

De plus, la bande passante utilisée entre le client et le serveur et fortement abrégée, car, lors d'une exécution, nous n'envoyons plus que les paramètres, au lieu d'envoyer la requête en entier.

Également, elles protègent contre les injections SQL, car la valeur passée en paramètre sera filtrée avant d'être envoyée.

Pour reprendre un exemple de mon projet, je vous propose de vous attarder sur une partie de ma classe *comment*. On voit qu'avant quoique ce soit, dans le constructeur, je viens préparer mes requêtes avec des paramètres. De plus, je précise déjà à ma requête la façon dont je veux recevoir les données.

```php
class Comment
{
    private $psGetComments = null;
    private $psGetCommentsByIdSpot = null;
    // [...]

    /**
     * Constructor
     * The constructor prepares all the SQL queries of the object,
     * in order to optimize the performance.
     */
    function __construct()
    {
        $sqlGetComments = "SELECT * FROM comments LIMIT :limit";
        $sqlGetCommentsByIdSpot = "SELECT * FROM comments c
                  LEFT JOIN users u on c.idUser = u.idUser
                  WHERE idSpot = :idSpot
                  ORDER BY c.createdAt ASC LIMIT :limit";
        // [...]

        $this->psGetComments = DBConnection::prepare($sqlGetComments);
        $this->psGetComments->setFetchMode(PDO::FETCH_ASSOC);

        $this->psGetCommentsByIdSpot = DBConnection::prepare($sqlGetCommentsByIdSpot);
        $this->psGetCommentsByIdSpot->setFetchMode(PDO::FETCH_ASSOC);
        // [...]
    }


    // [...]
}
```

Pour sécuriser un maximum mon projet, j'ai utilisé les Bound Parameters. Bound Parameters serait le terme générique, mais dans notre cas, pour MySQL, la fonction et le terme associé est Bind Parameters. En utilisant cette fonction, nous pouvons lier les paramètres d'une requête SQL préparés à nos vraies valeurs. De plus, la fonction *BindParam* nous permet de rajouter un filtre supplémentaire lors de la liaison des paramètres. En utilisant les mots clefs *PARAM_STR*, *PARAM_INT* ou encore *PARAM_BOOL*, on peut filtrer par rapport à un type de variable attendu. Dans ma classe *comment*, en plus de l'utilisation des *BindParam*, j'utilise un *try catch* qui me permet de gérer les éventuelles erreurs. En cas d'erreur, je créer une exception et j'enregistre l'erreur dans mes logs.

```php
class Comment
{
    // [...]

    /**
     * Get all comments order by grade.
     * @param   idSpot  The limite of the query, default : 10
     * @param   limit   The idSpot of the comments
     * @return  result  An associative array of all the comments
     */
    public function GetCommentsByIdSpot($idSpot, $limit = 10)
    {
        $result = null;

        try {

            $this->psGetCommentsByIdSpot->bindParam(':idSpot', $idSpot, PDO::PARAM_INT);
            $this->psGetCommentsByIdSpot->bindParam(':limit', $limit, PDO::PARAM_INT);
            $this->psGetCommentsByIdSpot->execute();

            $result = $this->psGetCommentsByIdSpot->fetchAll();
        } catch (PDOException $e) {

            error_log($e->getMessage());
            throw new Exception($e->getMessage());
        }

        return $result;
    }

    // [...]
}
```

6.3 Récupération des spots pour la map

Pour afficher les spots sur la map, il était impossible pour moi de récupérer tous les spots de toilettes de ma base de données et de les afficher. Cela prenait trop de ressource au serveur et au client. Pour pouvoir avoir une utilisation de ressource réduite, j'ai décidé de ne récupérer que les spots que l'utilisateur voyait à l'écran. De plus, j'ai créé un script qui enlève tous les marqueurs des spots qui ne sont plus affichés sur la map. Pour pouvoir récupérer les spots

dans un carré spécifique, j'ai utilisé le mot clef *BETWEEN* de MySQL, qui m'a permis de limiter la récupération des spots selon leurs positions.

```
SELECT * FROM spots WHERE lat BETWEEN :latMin AND :latMax AND lon BETWEEN :lonMin AND :lonMax LIMIT :limit
```

Grâce à Mapbox, j'ai pu récupérer aisément les quatre points de coordonnées qu'il me fallait. C'est-à-dire la latitude maximum, la latitude minimum, la longitude maximum et la longitude minimum. Pour ne pas devoir rafraichir la page à chaque fois que l'utilisateur bougeait la map, j'ai utilisé un appel AJAX. Ce qui m'a permis de rentrer dans le contrôleur de façon asynchrone, et de rafraichir les spots depuis le *callBack* de la fonction *success*, qui est systématiquement appelé lorsque l'appelle à la page asynchrone a réussi.

```
/**
 * @summary     Get the list of all the spot
 */
function GetSpots() {
    let bboxScreen = map.getBounds();

    let latMin = bboxScreen._sw.lat;
    let latMax = bboxScreen._ne.lat;

    let lonMin = bboxScreen._sw.lng;
    let lonMax = bboxScreen._ne.lng;

    $.ajax({
        url: "../controller/getSpotsAsMarkers.php",
        type: "POST",
        data: {
            latMin: latMin,
            latMax: latMax,
            lonMin: lonMin,
            lonMax: lonMax,
        },
        success: function (result) {
            let spots = JSON.parse(result);
            ShowSpots(spots);
        },
    });
}
```

6.4 Barre de recherche

Pour l'implémentation de la barre de recherche, je devais trouver un moyen de faire une recherche sur ma table *spots* par rapport aux noms, adresses et détails des spots. Pour ce faire, le premier moyen que j'ai trouvé pour y parvenir était d'utiliser l'opérateur *LIKE*. Il permet d'effectuer une recherche selon des critères donnés. Par exemple, il est possible de rechercher les données dont la valeur d'une colonne commence par tel caractère.

Le problème avec l'opérateur *LIKE* est qu'il est réservé à certaines utilisations. C'est-à-dire que cet opérateur va effectuer un scan complet de la colonne qui est filtré, et va rendre extrêmement lente la requête. Le second moyen que j'ai trouvé pour faire une recherche de façon optimiser, était les opérateurs *MATCH AGAINST*.

L'opérateur *MATCH AGAINST* n'a pas le même problème que l'opérateur *LIKE*. À savoir, qu'au lieu de faire un scan complet sur les colonnes concernées. *MATCH AGAINST* va faire une recherche dite *FULL TEXT* sur un index *FULL TEXT*. Disons qu'en indexant les champs testés au préalable avec un index *FULL TEXT*, la base de données arrange par ordre alphabétique tous ces mêmes champs. De ce fait, la base de données pourra faire une recherche dichotomique. La recherche dichotomique compare l'élément cherché par la valeur au milieu du dictionnaire de données. Si les deux valeurs sont égales, la recherche est accomplie. Si la valeur chercher est plus petite il va réeffectuer cette recherche dans la partie inférieure du dictionnaire, autrement dans la partie supérieure. En bref, c'est plus efficace.

```
SELECT * FROM spots WHERE MATCH(name, address, detail) AGAINST(:filter IN BOOLEAN MODE)
    ORDER BY CASE WHEN name = "" THEN 1 ELSE 0 end, name asc LIMIT :from, :offset;
```

Concernant le *ORDER BY*, j'ai voulu trier par ordre alphabétique des noms, mais malheureusement faire un simple *ORDER BY* sur le *name* ne suffisait pas. C'est-à-dire que lorsque je triais par les noms, c'étaient les champs vides qui étaient envoyés en premier. Malheureusement, MySQL ne supporte pas encore l'opérateur *NULL LAST*, alors j'ai été contraint d'utiliser l'opérateur *CASE WHEN … THEN … ELSE ... END*, comme ci-dessus, pour pouvoir afficher les spots ayant un nom vide en dernier.

Pour ne pas avoir à rafraichir la page constamment à chaque recherche de l'utilisateur, j'ai utilisé un appelle asynchrone, grâce à de l'AJAX, qui appelle le contrôleur pour obtenir les

spots. Dans le *callBack* nommé *success*, j'ai fait en sorte de rafraichir les spots qui étaient affichés.

```
/**
 * @summary     Asynchronously calls the controller to recover
 *              the spots according to the inserted filter.
 */
async function GetSpotsFiltered() {
    let filter = document.getElementById("searchBar").value;
    let from = currentPage * itemsPerPage - (itemsPerPage - 1) - 1;
    if (from < 0) from = 0;
    let offset = itemsPerPage;

    await $.ajax({
        url: "../controller/getSpotsFiltered.php",
        type: "POST",
        data: {
            filter: filter,
            from: from,
            offset: offset,
            onlyPopular: onlyPopular,
        },
        success: function (result) {
            if (result == "") result = "There are no spots that match your specifications.";
            document.getElementById("containerSpots").innerHTML = result;
        },
    });
}
```

6.5 Pagination

Pour gérer la pagination de la page *spots*, il me fallait plusieurs données :

- Le nombre maximum de spots par page
- La page courante
- Le nombre total de pages

Pour récupérer le nombre maximum de spots par page, j'ai simplement fait une constante de ce nombre, dans notre cas six.

Pour la gestion de la page courante, j'ai simplement utilisé une variable qui subissait une incrémentation ou une décrémentation, selon l'action de l'utilisateur. Évidement cette variable est soumise régulièrement à des tests pour pas qu'elle dépasse les valeurs limites, entre autres, être inférieur à un ou être supérieur au nombre total de pages. Voici les diverses fonctions que j'ai utilisées pour gérer la page courante.

```
currentPage = 1;
itemsPerPage = 6;
numberPage = await GetNumberPage();


/**
 * @summary     Go to next page, if possible
 */
function NextPage() {
    if (currentPage < numberPage) {
        SetPage(currentPage + 1);
    }
}

/**
 * @summary     Go to previous page, if possible
 */
function PreviousPage() {
    if (currentPage > 1) {
        SetPage(currentPage - 1);
    }
}

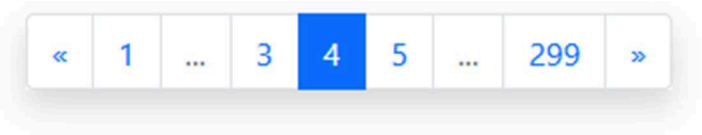
/**
 * @summary     Set the current page.
 * @param {number}  page    The number of the page we want to go
 */
function SetPage(page) {
    currentPage = parseInt(page);

    if (currentPage < 1) currentPage = 1;
    else if (currentPage > numberPage) currentPage = numberPage;

    GetSpotsFiltered();
}
```

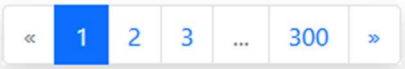
Pour récupérer le nombre total de pages j'ai effectué un simple appel AJAX sur un contrôleur qui me renvoie le nombre de page maximum, et prenant compte des différents filtres. Je m'abstiens de vous montrer cette fonction, car elle reste assez simple pour un appel AJAX.

Pour la navigation qui permet à l'utilisateur de se balader entre les pages, j'ai choisi de créer une navigation de ce type-là :



Il y a en tout sept boutons qui doivent respecter certains états. Ci-dessous, tous les boutons de façon détaillés :

- Le bouton page précédente mène à la page précédente. Il est généralement activé, mais il peut être désactivé si l'utilisateur se trouve à la première page.
- Le bouton page minimum mène à la première page. Il ne peut pas être désactivé, mais il peut devenir invisible. Il l'est lorsque d'autres boutons en viennent à afficher la page minimum. Par exemple lorsque l'utilisateur se trouve sur les premières pages :



- Les boutons de séparation ne font rien. Ils sont là pour montrer qu'il y a un certain nombre de boutons entre le bouton de la première page et le bouton de sélection de pages. Ils peuvent être invisibles, s'il n'y a pas d'écart entre le bouton de première page et celui de sélection de pages, comme au-dessus.
- Les boutons de sélection de pages s'affichent en priorité. Ils peuvent être invisibles si le nombre de pages est inférieur à leurs nombres. Le bouton de sélection de pages actif, c'est-à-dire en bleu, montre la page sur laquelle est l'utilisateur.



- Le bouton page maximum mène à la dernière page. Il ne peut pas être désactivé, mais il peut devenir invisible, de la même façon que le bouton première page. Il l'est lorsque d'autres boutons en viennent à afficher la page maximum. Par exemple lorsque l'utilisateur se trouve sur les dernières pages :
- Le bouton page suivante mène à la page suivante. Il est généralement activé, mais il peut être désactivé si l'utilisateur se trouve à la dernière page.

J'ai séparé en deux parties le rafraichissement des boutons, la partie algorithme et la partie affichage. Je détaillerai uniquement la partie algorithme, car c'est de loin la plus intéressante. L'algorithme se fait en plusieurs étapes de traitement.

Avant de commencer l'algorithme, on initialise les valeurs des boutons à *false*. Si les boutons sont toujours à *false* lorsqu'ils doivent s'afficher, c'est qu'ils doivent être invisibles ou désactivés. On va aussi initialiser la valeur des boutons de sélection de pages. Ils sont définis de la valeur de la page moins un, à la valeur de la page plus un.

Lorsqu'on commence l'algorithme, on va, en premier, définir l'état des boutons précédent et suivant, s'ils sont activés ou non. Ils sont actifs seulement s'il un appui sur l'un d'eux ne dépasse pas le nombre de pages.

En deuxième, on vient définir la visibilité des boutons séparateurs. Le premier séparateur est visible uniquement si l'écart entre la page actuelle et la page minimum est supérieur à trois. Et le deuxième bouton séparateur est uniquement visible s'il a un écart de trois avec la page actuelle et la page maximum.

En troisième, on fixe les valeurs des boutons des pages minimum et maximum. La page minimum étant toujours à un, on la fixe à un et la page maximum on la fixe au nombre de pages maximum.

En quatrième, on vérifie la fiabilité du texte des boutons. C'est-à-dire que si la page courante est égale à un, on décale la valeur des boutons d'un. C'est pour ne pas se retrouver avec un bouton ayant comme valeur zéro. On fait exactement la même chose, mais de façon opposée si la page courante est égale à la page maximum.

En cinquième, on va vérifier la validité des boutons. De façon à ce qu'il n'y ait pas trop de boutons pour le nombre de pages.

Et finalement, on rafraichit tous les boutons avec les valeurs que l'on vient de calculer.

L'algorithme final ressemble à celui-ci :

```
function UpdatePagination() {
    // initialise btn, false unactive/invisible, true/number the opossite
    // [...]

    // set the text of the button
    let textBtn1 = currentPage - 1;
    let textBtn2 = currentPage;
    let textBtn3 = currentPage + 1;

    // set activation/visiblity, btn previous/next or separator
    if (currentPage > 1) buttonPrevious = true;
    if (currentPage < numberPage) buttonNext = true;
    if (currentPage > 1 + 2) buttonSeparator1 = true;
    if (currentPage < numberPage - 2) buttonSeparator2 = true;

    // set the value of the button min & max
    buttonMin = 1;
    if (numberPage > 1) buttonMax = numberPage;

    // if the current page is 1, we need to add 1
    if (currentPage == 1) {
        textBtn1++; textBtn2++; textBtn3++;
    }
    if (currentPage == numberPage) {
        textBtn1--; textBtn2--; textBtn3--;
    }

    // before valide the button page, check if they can be alowed
    if (textBtn1 > 1) buttonPage1 = textBtn1;
    if (numberPage > 2) buttonPage2 = textBtn2;
    if (textBtn3 < numberPage) buttonPage3 = textBtn3;

    UpdateButtons(); //[...]
}
```

7 Tests

7.1 Méthodologie de test

Dans le cadre de mon TPI, j'ai fait des tests de façon quotidienne. Ils m'ont été très utiles pour voir l'avancement de mon travail et exprimé si les besoins de l'énoncé étaient en train d'être remplis. J'avais aussi pensé à le faire uniquement à la fin de la conception de mon projet, mais je n'ai pas opté pour cette façon de faire, car je trouvais que cette méthode néglige trop les tests intermédiaires. Les tests que j'effectuais ont été penser avant le commencement du développement de mon site web. Ils se basent sur le *Product Backlog* que j'ai organisé au départ de ce projet. Je les ai déjà énumérées dans la partie planification de mon projet.

Pour effectuer mes tests, je n'ai pas voulu utiliser un Framework externe. C'est quelque chose que je ne maitrise pas assez pour l'inculquer dans la méthodologie de tests que j'ai appliquée ici. J'ai préféré me référer à des tests que je fais de façon manuellement sur le code et sur le site web.

7.2 Scénarios de test

Les scénarios de test sont basés sur les *Product Backlog*. Chacun de ces tests est séparé en quatre parties distinctes. Le titre qui représente en bref le test. La tâche associée qui reprend l'*User Story* du *Product Backlog* qui est en train d'être testé. La situation qui exprime la démarche qui doit être effectuée pour faire le test. Et le statut dans lequel figure l'état du test, s'il est réussi ou non.

Titre	**1 – Récupération des données**
Tâche associée	1 – Récupération des données
Situation	La base de données est remplie de données fictives ou non. C'est-à-dire au minimum : • 10 spots • 5 utilisateurs de différents rôles • 0 à 3 commentaires par spot
Statut	✓

Titre	**2 – Élaboration de la base de données**
Tâche associée	2 – Élaboration de la base de données
Situation	La base de données est conforme et peut être utilisée simplement pour toutes les fonctionnalités du site. Il se réfère au MCD mis en place.
Statut	✓

Titre	**3.1 – Implémentation de la map, affichage**
Tâche associée	3 – Implémentation de la map
Situation	En tant utilisateur quelconque. J'ai la possibilité de voir une map sur le site web. Peu importe la grandeur de mon écran, je peux voir simplement les informations que donne la map. L'utilisateur peut s'il en a l'envie agrandir la map lorsqu'il appuie sur le coin de la map, quand il se trouve dans la page d'ajouts de spot.
Statut	✓

Titre	**3.2 – Implémentation de la map, fonctionnalité**
Tâche associée	3 – Implémentation de la map
Situation	En tant qu'utilisateur connecter ou non. Je peux simplement utiliser la map. Lorsque j'effectue un appui continu sur la map, je suis capable de la déplacer pour aller voir des informations qui sont de base hors cadre. Lorsque je scroll, ou pince sur téléphone, sur la map. Je peux zoomer ou dé-zoomer, de manière à voir plus ou moins de détail.
Statut	✓

Titre	**4 – Affichage des spots populaires**
Tâche associée	4 - Affichage des spots populaires
Situation	En tant qu'utilisateur connecté ou non. Je peux, dans la page d'accueil, voir les spots les plus populaires. L'utilisateur peut voir les informations suivantes : • Son nom • Son adresse • Sa popularité • Ses images Ils sont triés par ordre des popularités. Cliquer dessus nous renvoie sur leur page d'information.
Statut	✓

Titre	**5.1 – Recherche d'un spot, existant**
Tâche associée	5 – Recherche d'un spot
Situation	En tant qu'utilisateur connecter ou non. Dans la page spot, lorsque je recherche un spot, par son nom, ou son adresse. Les spots les plus pertinents s'affichent sous forme de liste. Par exemple, si je rentre « Street Workout » les spots qui contiennent les mots « Street » ou « Workout » s'affichent. Rechercher un post depuis la page d'accueil, me renvoie sur la page des spots.
Statut	✓

Titre	**5.2 – Recherche d'un spot, inexistant**
Tâche associée	5 – Recherche d'un spot
Situation	En tant qu'utilisateur connecter ou non. Lorsque je recherche un spot, par son nom, ou son adresse, qui n'existe pas. Un message me disant qu'aucun spot avec ces propriétés n'existe. Par exemple, si je rentre « uneruequinexistepas 20 ». Aucun n'éléments serra présent sur la liste et un message indiquera qu'aucun élément ne convient au critère demandé.
Statut	✓

Titre	**6.1 – Affichage des spots sous forme de liste, affichage**
Tâche associée	6 – Affichage des spots sous forme de liste
Situation	En tant qu'utilisateur connecter ou non. Dans la page de spot, je peux voir la liste de tous les spots s'afficher. Ils contiennent tous : • Une image • Un nom • Une adresse • Une popularité. Un cœur est affiché sur chaque spot.
Statut	✓

Titre	**6.2 – Affichage des spots sous forme de liste, interaction**
Tâche associée	6 – Affichage des spots sous forme de liste
Situation	En tant qu'utilisateur connecter ou non. Dans la page spot, je peux cliquer sur le nom d'un des spots, pour que cela me redirige sur sa page d'information. Je peux cliquer sur l'image d'un des spots, pour que cela me redirige sur sa page d'information.
Statut	✓

Titre	**6.3 – Affichage des spots sous forme de liste, interaction connecter**
Tâche associée	6 – Affichage des spots sous forme de liste
Situation	En tant qu'utilisateur connecter. Je peux cliquer sur le cœur vide d'un des spots, pour aimer le spot. Je peux cliquer sur le cœur rempli d'un des spots, pour ne plus aimer le spot.
Statut	✓

Titre	**6.4 – Affichage des spots sous forme de liste, pagination**
Tâche associée	6 – Affichage des spots sous forme de liste
Situation	En tant qu'utilisateur connecter ou non. Je peux voir un maximum de six spots par pages. Lorsque je clique sur le bouton avec un « < » dessus, je peux passer à la page précédente. Lorsque je clique sur le bouton avec un « > » dessus, je peux passer à la page suivante. Un indicateur me montre à quelle page je suis, et le nombre maximum de spots, par exemple : (1 à 6 de 132 Spots).
Statut	✓

Titre	**7.1 – Filtrage des spots, popularité**
Tâche associée	7 – Filtrage des spots
Situation	En tant qu'utilisateur connecter ou non. Dans la page spots, lorsque je clique sur un bouton avec une icône de filtre dessus, je peux filtrer les spots par popularité. Le clique effectuer, les spots se recharge et s'affiche de manière souhaiter. Un deuxième clique sur le filtre désactive de la même façon le filtre.
Statut	✓

Titre	**7.2 – Filtrage des spots, adresse**
Tâche associée	7 – Filtrage des spots
Situation	En tant qu'utilisateur connecter ou non. Lorsque je clique sur l'adresse d'un des spots, je peux filtrer les spots par rapport à l'adresse sur laquelle j'ai cliqué (uniquement les spots issus de cette région s'affichent). Le clique effectuer, les spots se recharge et s'affiche de manière souhaiter.
Statut	✓

Titre	**8.1 – Gestion de popularité, connecter**
Tâche associée	8 – Gestion de popularité
Situation	En tant qu'utilisateur connecter. Lorsque je me trouve dans la page d'information d'un spot. Je peux voter pour un post avec un système d'étoile, en cliquant sur le nombre d'étoiles que je veux mettre au spot. Je vois un cœur plein ou vide si j'ai aimé ou non le spot. Je peux aimer le spot en cliquant sur le cœur vide du spot. Je peux arrêter d'aimer le spot en cliquant sur le cœur plein du spot.
Statut	✓

Titre	**8.2 – Gestion de popularité, non connectée**
Tâche associée	8 – Gestion de popularité
Situation	En tant qu'utilisateur non connecté. Lorsque je me trouve dans la page d'information d'un spot. Je ne peux pas voter pour un post, en cliquant sur le nombre d'étoiles que je veux mettre au spot, car cela me redirige vers la connexion de compte. Je vois un cœur vide, car je ne suis pas connecté.
Statut	✓

Titre	**9.1 – Ajout d'un spot, connecter**
Tâche associée	9 – Ajout d'un spot
Situation	En tant qu'utilisateur connecter. Lorsque je suis sur la page d'ajout d'un spot et que je rentre, au minimum, tous les champs obligatoires. C'est-à-dire : • Sa localisation • Son image Lorsque je clique sur le bouton « envoyer », je suis redirigé sur la page des spots, avec un message de succès.
Statut	✓

Titre	**9.2 – Ajout d'un spot, non connecté**
Tâche associée	9 – Ajout d'un spot
Situation	En tant qu'utilisateur non connecté. L'onglet ne s'affiche pas dans la navigation. Et je ne peux pas accéder à la page d'ajout de spot.
Statut	✓

Titre	**9.3 – Ajout d'un spot, champs incomplets**
Tâche associée	9 – Ajout d'un spot
Situation	En tant qu'utilisateur connecter. Lorsque je suis sur la page d'ajour d'un spot et que je ne rentre pas, au minimum, les champs obligatoires. Lorsque je clique sur le bouton « envoyer », je suis redirigé sur la même page, avec mon formulaire en *sticky*, et avec un message d'erreur.
Statut	✓

Titre	**10 – Affichage des spots sous forme de marqueur.**
Tâche associée	10 – Affichage des spots sous forme de marqueur.
Situation	En tant qu'utilisateur connecter ou non. Lorsque je suis sur la page carte, je peux voir une carte avec des marqueurs qui symbolise des spots. Lorsque clique sur un des marqueurs, une pop-up apparait avec le nom du spot. Si je clique sur le spot, je suis redirigé sur la page avec ces informations.
Statut	✓

Titre	**11 – Affichage des informations d'un spot**
Tâche associée	11 – Affichage des informations d'un spot
Situation	En tant qu'utilisateur connecter ou non. Sur la page d'information d'un spot, je peux voir toutes les informations d'un spot. C'est-à-dire : • Son nom • Son adresse • Sa popularité • Les commentaires • Ses images Si des informations manquent, elles sont signalées d'un message.
Statut	✓

Titre	**12 – Gestions des spots**
Tâche associé	12 – Gestions des spots
Situation	En tant qu'utilisateur connecté en administrateur. Sur la page d'information d'un spot. Je peux modifier, supprimer et valider un spot. Pour effectuer une action, je peux cliquer sur l'un des icônes représentant une des actions possibles. Lorsque je modifie le spot, je suis redirigé sur une autre page qui me permet de modifier le spot via un formulaire. Lorsque je supprime le spot, je suis redirigé sur la page des spots, le spot supprimé n'est plus présent dans la liste. La validation est possible que si le post n'est pas encore validé.
Statut	✓

Titre	**13 – Gestions des commentaires**
Tâche associée	13 – Gestions des commentaires
Situation	En tant qu'utilisateur connecté en administrateur. Sur la page d'information d'un spot. Je peux modifier et supprimer un commentaire. Pour effectuer une action, je peux cliquer sur l'une des icônes qui se situe à côté de chaque commentaire, représentant sois la suppression ou la modification du commentaire. Lorsque j'efface un commentaire, la page se recharge sans le commentaire en question. Lorsque je modifie un commentaire, cela m'emmène sur une autre page qui me permet de modifier le commentaire.
Statut	✓

Titre	**14.1 – Gestion de son profil, modification**
Tâche associée	14 – Gestion de son profil
Situation	En tant qu'utilisateur connecter. Je peux afficher toutes mes informations via la page de compte. En cliquant sur l'icône de modification, je suis redirigée sur une page contenant un formulaire de modification de mon compte. Lorsque j'ai fini de modifier mes informations, en cliquant sur le bouton « envoyer ». Je suis redirigé sur la page d'information de mon compte. Les nouvelles informations sont présentes.
Statut	✓

Titre	**14.2 – Gestion de son profil, déconnecter**
Tâche associée	14 – Gestion de son profil
Situation	En tant qu'utilisateur connecter. Je peux afficher toutes mes informations via la page de compte. En cliquant sur l'icône de déconnexion, je peux me déconnecter. Je suis alors redirigé sur la page d'accueil.
Statut	✓

Titre	**14.3 – Gestion de son profil, suppression**
Tâche associée	14 – Gestion de son profil
Situation	En tant qu'utilisateur connecter. Je peux afficher toutes mes informations via la page de compte. En cliquant sur l'icône de suppression de compte, une pop-up s'affichera et pour confirmer mon action. Si je la confirme, mon compte sera définitivement supprimé, mais pas mes commentaires. À la suite de la suppression de mon compte, je suis redirigé vers la page d'accueil.
Statut	✓

Titre	**15.1 – Gestion des utilisateurs, affichage**
Tâche associée	15 – Gestion des utilisateurs
Situation	En tant qu'utilisateur connecté en administrateur. J'ai accès à une page qui liste tous les utilisateurs (la page utilisateurs) Je peux filtrer les utilisateurs en fonction de leurs pseudos, par ordre ascendant ou descendant. À l'aide d'une barre de recherche, je peux rechercher par adresse email les utilisateurs. Après une recherche, les utilisateurs avec les résultats les plus pertinents sont affichés d'abord.
Statut	✓

Titre	**15.2 – Gestion des utilisateurs, affichage**
Tâche associée	15 – Gestion des utilisateurs
Situation	En tant qu'utilisateur connecté en administrateur. Dans la page utilisateurs, pour chaque utilisateur, je peux les supprimer ou les modifier. Lorsque je supprime un utilisateur, une pop-up apparait pour valider mon choix. Si je valide mon action, la liste se rafraichit, sans l'utilisateur en question. Lorsque je modifie un utilisateur, je suis redirigé sur une page qui me permettra de modifier l'utilisateur via un formulaire.
Statut	✓

Titre	**16.1 – Gestion des rôles, non connectée**
Tâche associée	16 – Gestion des rôles
Situation	En tant qu'utilisateur non connecté. Je n'ai pas accès au page qui est exclusivement fait pour les administrateurs. Je ne peux pas effectuer des actions que seul les identifier peuvent faire.
Statut	✓

Titre	**16.2 – Gestion des rôles, connecter**
Tâche associée	16 – Gestion des rôles
Situation	En tant qu'utilisateur connecter. Je n'ai pas accès au page qui est exclusivement fait pour les administrateurs. Je peux effectuer des actions que seul les identifier peuvent faire.
Statut	✓

Titre	**16.3 – Gestion des rôles, administrateur**
Tâche associée	16 – Gestion des rôles
Situation	En tant qu'utilisateur connecter en tant qu'administrateur. J'ai accès au page qui est exclusivement fait pour les administrateurs. Je peux effectuer des actions que seul les identifier peuvent faire.
Statut	✓

Titre	**17.1 – Création d'un compte, redirection**
Tâche associée	17 – Création d'un compte
Situation	En tant qu'utilisateur non connecté. Je peux, via le bouton « créer un compte », me créer un compte. En cliquant sur le bouton, je suis redirigé sur la page de création de comptes, qui me permet, grâce à un formulaire de me créer un compte. Le bouton ne s'affiche pas si je suis connecté.
Statut	✓

Titre	**17.2 – Création d'un compte, formulaire**
Tâche associée	17 – Création d'un compte
Situation	En tant qu'utilisateur non connecté. Je peux me créer un compte en cliquant sur le bouton « envoyé » uniquement si tous les champs obligatoires dans le formulaire d'inscription sont remplis, c'est-à-dire : • Pseudo • Email • Mots de passe 2x Dans ce cas, je suis redirigé vers la page d'accueil, connecté, avec un message de succès. Dans le cas contraire, je suis redirigé sur la même page, avec un message d'erreur.
Statut	✓

Titre	**18.1 – Connexion au site, succès**
Tâche associée	18 – Connexion au site
Situation	En tant qu'utilisateur non connecté. J'ai accès à un bouton de connexion directement dans la navigation. En cliquant dessus je suis redirigé sur la page de connexion, avec un formulaire de connexion. Si je remplis le formulaire avec les bonnes données de mon compte et que je clique sur le bouton « envoyé », je suis redirigé sur la page d'accueil.
Statut	✓

Titre	**18.2 – Connexion au site, erreur**
Tâche associée	18 – Connexion au site
Situation	En tant qu'utilisateur non connecté. J'ai accès à un bouton de connexion directement dans la navigation. En cliquant dessus je suis redirigé sur la page de connexion, avec un formulaire de connexion. Si je remplis le formulaire avec de mauvaises données de mon compte et que je clique sur le bouton « envoyé », je suis redirigé sur la même page avec un message d'erreur.
Statut	✓

7.3 Évolution des tests

L'évolution des tests est un tableau qui représente l'avancement des tests sur toute la durée du projet. Sur ce tableau, y est présent uniquement le statut des tests.

Test Numéro	J0 03.mai	J1 04.mai	J2 05.mai	J3 06.mai	J4 10.mai	J5 11.mai	J6 12.mai	J7 17.mai	J8 18.mai	J9 19.mai	J10 20.mai
1	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
2	✗	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓
3.1	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓
3.2	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓
4	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓
5.1	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
5.2	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
6.1	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓
6.2	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓
6.3	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓
6.4	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
7.1	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
7.2	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓
8.1	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
8.2	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
9.1	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓
9.2	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
9.3	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓
10	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓
11	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓
12	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
13	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
14.1	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓
14.2	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
14.3	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
15.1	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
15.2	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
16.1	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
16.2	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
16.3	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
17.1	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
17.2	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
18.1	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
18.2	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓

8 Conclusion

8.1 Difficultés rencontrées

Lors de la conception de mon projet, je n'ai pas particulièrement rencontré de difficultés. Mais il y a quand même quelques points sur lesquelles je pensais être assez efficace, mais qu'au contraire, j'ai fini par prendre bien plus de temps que nécessaire. Par exemple, on peut nommer les points suivants :

Lors de la conception du diagramme de la base de données, je pensais avoir pris en compte toutes les demandes du cahier des charges, de façon à ce que mon diagramme puisse les remplir. Mais malheureusement pour moi, j'ai fini par devoir le changer à deux reprises. La première fois lorsque je faisais le système de votes et de "j'aime". J'ai remarqué que je ne stockais que la note moyenne du spot. Au premier regarde, cela ne semble pas être dérangeant, mais comment faire une moyenne si je n'ai pas le nombre de notes et les notes que l'on donne au spot. De plus faire de cette façon m'empêchait de vérifier si un utilisateur avait déjà voté ou non. J'ai été alors contraint de faire une nouvelle table. Ce que j'ai fait n'a, heureusement, pas trop désorganiser ma base de données. J'ai, enfaite, créer une nouvelle table qui faisait une liaison entre l'utilisateur et le spot. Cette table je l'ai nommé vote, elle a un champ rate qui enregistre la note, et elle utilise *idSpot* et *idUser*, qui sont des clefs étrangères, comme clef primaire. La seconde fois où j'ai dû changer la base de données, c'est lorsque j'ai remarqué que j'avais oublié le champ *validate* dans la table spot. Il m'a suffi, cette fois-ci, de simplement rajouter le champ.

Pour la pagination, je ne pensais pas que cela me prendrait autant de temps. En réalité je pensais même pouvoir récupérer une pagination préfaite de ma librairie de CSS, *Bootstrap*. J'ai alors dû élaborer tout un algorithme qui m'a permis de faire cette pagination. Il faut dire aussi que le type de pagination que j'ai pris n'était pas non plus le plus simple.

Lorsqu'avec mon professeur de TPI, M. Garcia, nous avons essayé d'implémenter des fichiers *.htaccess*, nous avons pris un certain temps. Ce qui m'a mis un peu en retard pour la suite. De plus, nous avons finalement par utiliser cette façon de faire. Nous avons préféré protéger les fichiers avec des traitements PHP.

Pour ce qui est de la planification, je ne l'ai pas particulièrement ratée, mais je pense que j'aurais pu être un peu plus précis par rapport à la priorité des tâches. En effet, au milieu de mon TPI, je me suis rendu compte que certaines tâches auraient dû être fait plus tôt ou plus tard. Mis à part ce souci, je pense avoir bien prévu et respecté mon planning prévisionnel. On peut d'ailleurs remarquer qu'il n'y a pas tant de différence entre le réalisationnel et le prévisionnel.

8.2 Amélioration possible

Je suis assez satisfait du travail que j'ai fourni en général, mais il y a tout de même des points que j'aurais pu améliorer avec plus de temps. Je pense qu'il y a quatre points qui auraient pu être révisés.

La sécurité générale de l'application n'est pas mauvaise, mais je pense qu'il y a quelque aspect qui n'aurait pas été de trop. Par exemple lors de la connexion d'un utilisateur, j'aurais pu ajouter un *captcha*, de même dans l'ajout d'un spot. Pour ce qui est de l'accessibilité des fichiers, je pense avoir fait le nécessaire, mais nous ne sommes jamais à l'abri de tous, alors plusieurs vérifications n'auraient pas dérangé. Je pense aussi que limité l'accès au fichier de configuration en les mettant en dehors du dossier *www* aurait été une bonne idée.

Avec plus de temps, j'aurais pu travailler sur l'aspect design du site web. Je n'ai malheureusement pas eu le temps de travailler sur un logo d'application. Et j'ai gardé les couleurs de base que me proposais *Boostrap*, ce qui n'est pas très réjouissant. Malgré le fait que j'ai utilisé une libraire *CSS*, le site web n'est pas encore complètement responsive. Et avec plus de temps, je pense pouvoir aisément le rendre responsive. Il me suffirait d'ajouter quelques règles de *CSS*. Je pense que j'aurais pu aussi prendre en compte la taille des images que les utilisateurs ajoutent dans le site web, en effet, il n'y a pas de formatage des images pour le moment.

Un des points que je n'ai pas eu le temps de faire, mais que je pensais faire de base, était l'ajout pour chaque utilisateur d'une image de profil. J'aurais voulu, si possible, donner la possibilité à l'utilisateur de mettre une image de profil qui lui appartient et de lui laisser le pouvoir de la modifier. Mais, malheureusement, je n'ai pas eu le temps de le faire, car j'ai remarqué que la modification d'une image de profil me prendrait bien plus de temps de ce que je pensais, et puisque ce n'était pas un point qui était explicitement demandé dans mon cahier des charges, j'ai préféré ne pas le faire.

Un point qui relève de l'architecture de mon projet et non des fonctionnalités de celui-ci est la méthode de gestion des vues. J'aurais voulu si possible pouvoir créer un routeur, qui aurait géré la distribution des routes, c'est-à-dire de mes vues, ainsi que la disponibilité des pages, c'est-à-dire leurs sécurités. Cela aurait extrêmement facilité la lisibilité de mon projet par rapport à la navigation. De plus un routeur aurait fortement aidé à la centralisation de certaines de mes actions. Si l'on reprend l'exemple de la sécurité, celle-ci aurait été gérer uniquement dans mon routeur, au lieu d'être présent dans presque la totalité de mes pages.

8.3 Bilan personnel

Lors de ces jours dix derniers jours, j'ai eu la chance de travailler sur un projet qui me tenait à cœur. Le développement web ayant toujours été ma façon de programmé préféré il était normal que je travaille sur un tel projet pour mon projet final de CFC. Lors de ce projet, j'ai pu approfondir quelques connaissances que je ne maitrisais pas parfaitement auparavant. La reddition d'un journal de bord est vraiment quelque chose qui changera ma façon de voir mes projets futurs. Écrire une bonne documentation n'a pas été quelque chose d'anodin, mais cela m'a donné beaucoup d'expérience dans le domaine. Je suis content que mon dernier projet de mon CFC soit un travail qui était concret à mes yeux. Finalement, arrivant à la fin de ce projet, je peux dire que je suis fier de moi et que j'ai donné tout ce que j'avais.

8.4 Remerciements

Je tiens également à remercier Monsieur Garcia pour son accompagnement et l'aide qu'il m'a fournie durant mon TPI.

9 Bibliographie

Lors de l'implémentation de mon projet et de ma documentation, j'ai utilisé les différentes ressources ci-dessous pour m'aider à réaliser mon projet :

- « Qu'est-ce qu'un diagramme de Gantt ? », Gantt.com, le 05.2021, https://www.gantt.com/fr/
- « IPDRCE La méthode en six étapes », fichier PDF, le 05.2021, https://www.frecem.ch/fileadmin/user_upload/Formation/Menuisier-Ebeniste/CFC_Menuisier_Ebeniste/Classeurs_CIE/1_4_IPDRCE.pdf
- « Planification par la méthode des 6 étapes », fichier PDF, le 05.2021, https://www.afci-ju.ch/fichiers/Planification-par-la-mthode-des-6-tapes-15.pdf
- « Modèle-vue-contrôler », Wikipédia, le 05.2021, https://fr.wikipedia.org/wiki/Mod%C3%A8le-vue-contr%C3%B4leur
- « Mapbox GL JS », Mapbox, le 05.2021, https://docs.mapbox.com/mapbox-gl-js/api/
- « Add custom markers in Mapbox GL JS », Mapbox, le 05.2021, https://docs.mapbox.com/help/tutorials/custom-markers-gl-js/
- « PHP MySQL Prepared Statements », W3 Schools, le 05.2021, https://www.w3schools.com/php/php_mysql_prepared_statements.asp
- « PDOStatement::bindParam », PHP.net, le 05.2021, https://www.php.net/manual/fr/pdostatement.bindparam.php
- « Les requêtes préparées », PHP.net, le 05.2021, https://www.php.net/manual/fr/mysqli.quickstart.prepared-statements.php
- « SQL LIKE », SQL.sh, le 05.2021, https://sql.sh/cours/where/like
- « MATCH AGAINST », MariaDB, le 05.2021, https://mariadb.com/kb/en/match-against/
- « Full-Text Index Overview », MariaDB, le 05.2021, https://mariadb.com/kb/en/full-text-index-overview/
- « Recherche dichotomique », Wikipédia, le 05.2021, https://fr.wikipedia.org/wiki/Recherche_dichotomique
- « Which SQL query is better, MATCH AGAINST or LIKE? », Stack Overflow, le 05.2021, https://stackoverflow.com/questions/792875/which-sql-query-is-better-match-against-or-like
- « Window.localStorage », MDN Web Docs (Mozilla), le 05.2021, https://developer.mozilla.org/fr/docs/Web/API/Window/localStorage